



Тема ВКР: «Методика автоматической трансляции программного кода между языками с сохранением семантики безопасности»

ФИО студента: Ушаков Андрей Вячеславович

10.04.01 - Информационная безопасность

Программа: Безопасность информационных систем

Дата защиты 08.06.2026



Актуальность темы и проблематика исследования

После трансляции код может потерять защитные свойства исходного языка и кода.

Где возникает проблема?

- миграция legacy-кода;
- выпуск библиотек под разные языки;
- импортозамещение платформ;
- использование LLM для трансляции кода.

Главный риск

После трансляции код может функционально работать, но потерять защитные свойства исходного языка и кода.

Разрыв в существующих подходах:

Существующие подходы (Secure compilation / Translation validation / Verified compilation) в основном контролируют функциональную корректность, компиляцию или уязвимости после трансляции, но не дают прикладного механизма сравнения защитных свойств до и после трансляции высокоуровневого кода.

Пример потерь при трансляции с языка C# на язык Python

Гарантия в C#	Что происходит в Python
Статическая типизация	Требуется runtime-компенсация (из-за duck typing)
Перегрузка методов	Нужна диспетчеризация
Инкапсуляция, модификаторы доступа private/protected	Полноценной поддержки нет
readonly поля	Прямого аналога нет

Проблема ВКР: нужен контрольный слой, который показывает, какие инварианты безопасности сохраняются, компенсируются или теряются при трансляции.

Цель работы — повысить управляемость и предсказуемость изменений инвариантов безопасности программного кода при трансляции между языками за счёт разработки и экспериментальной проверки методики автоматической трансляции с контролируемым сохранением семантики безопасности.

Критерии достижения цели

Цель считается достигнутой, если...	Как проверяется / подтверждается
Разработана модель инвариантов безопасности и отпечатка безопасности	Дано определение инварианта, отпечатка безопасности и состояний инвариантов: поддерживается полностью / частично / не поддерживается
Инварианты классифицированы по характеру изменения при трансляции	Введены классы L1–L4: сохраняются, компенсируются, теряются как допустимое/недопустимое ослабление, появляются в целевом коде
Определены условия успешной трансляции	Проверяются функциональность, поведение, публичный API, сохранение L1, компенсация L2 и допустимость потерь L3
Разработана методика контролируемой трансляции	Представлен процесс: исходный код → отпечаток до → L1–L4 → решение по L3 → симплификация → трансляция и L2-компенсации → отпечаток после → сравнение
Реализован исследовательский прототип	Созданы симплификатор C#-кода и транслятор с языка C# на язык Python с компенсациями для L2-инвариантов
Проведена экспериментальная проверка	Методика сравнена с ручной трансляцией и LLM-трансляцией по функциональности, API, L1, L2, L3 и предсказуемости

1. Разработка модели защитных инвариантов программного кода и отпечатка безопасности.
2. Разработка классификации изменений защитных инвариантов при трансляции между языками.
3. Определение условий сохранения семантики безопасности и успешности трансляции.
4. Разработка методики автоматической трансляции с контролем отпечатка безопасности и допустимых ослаблений.
5. Разработка механизма симплификации исходного кода и реализация прототипа трансляции с языка C# на язык Python.
6. Проведение экспериментальной проверки методики на примере трансляции с языка C# на язык Python, включая сравнение с ручной трансляцией и трансляцией с помощью большой языковой модели (LLM).

Модель: инвариант безопасности и отпечаток безопасности

Инвариант безопасности — это свойство программного кода, языка или среды выполнения, нарушение которого может привести к ослаблению гарантий безопасности, появлению уязвимости или некорректному использованию программного интерфейса.

Примеры:

- проверка входных аргументов;
- типобезопасность;
- инкапсуляция;
- readonly-семантика;
- корректная обработка исключений;
- корректное управление ресурсами.

Источники инвариантов:

- спецификация языка
- CWE (CVE) / CERT / OWASP
- ГОСТ Р 56939 / ГОСТ Р 58412
- БДУ ФСТЭК
- контекст использования кода

Отпечаток безопасности — это структурированный набор инвариантов безопасности кодовой базы или отдельной сущности кода с указанием состояния каждого инварианта: поддерживается полностью, поддерживается частично или не поддерживается.

Примеры инвариантов безопасности

Инвариант	Источник	Значение в исходном коде
Проверка входных аргументов	код / CWE / CERT	Поддерживается
Типобезопасность	Спецификация C#	Поддерживается
Инкапсуляция	Спецификация C#	Поддерживается
readonly - семантика	Спецификация C#	Поддерживается

Классификация L1–L4

Класс	Что происходит при трансляции	Действие методики	Пример (для трансляции с C# на Python)
L1	сохраняется автоматически	контролировать сохранение	проверки входных аргументов
L2	напрямую не поддерживается, но может быть симитировано	сгенерировать компенсацию	типобезопасность, перегрузки
L3	не сохраняется или крайне трудно симитировать	зафиксировать допустимое или недопустимое ослабление	инкапсуляция, модификаторы доступа, readonly поля
L4	появляется в целевом коде	зафиксировать улучшение	безопасность строк при трансляции с C на Python

Важно

Классификация инвариантов безопасности зависит от пары исходный язык **A**, целевой язык **B**. Если для данного исходного языка **A** поменяется целевой язык **B**, то классификация инвариантов безопасности может быть другой. Так, например, для пары языков C#→Python типобезопасность имеет класс **L2**, а модификаторы доступа и инкапсуляции – **L3**; для пары языков C#→C++, типобезопасность и модификаторы доступа и инкапсуляция – **L1**

Важно

Инварианты безопасности L3 не являются ошибкой транслятора, а отражают ограничение целевого языка: они не могут быть полноценно воспроизведены штатными средствами целевого языка, как эквивалентные гарантии исходного языка. Например, инкапсуляция и модификаторы доступа при трансляции кода с языка C# на язык Python.

Формальная модель контроля семантики безопасности

Модель программы

Пусть программа P_A на языке A представлена как кортеж:

$$P_A = (AST_A, S_A, I_A), \quad (6)$$

где:

AST_A - синтаксическое дерево (abstract syntax tree) программы P_A ;

S_A - множество семантических свойств поведения программы P_A ;

I_A - множество свойств безопасности (инвариантов безопасности) программы P_A .

Оператор трансляции T программы (исходного кода) на языке A в программу (исходного кода) на языке B определяется как отображение:

$$T: P_A \rightarrow P_B, \quad (7)$$

где:

P_A - программа на языке A ;

P_B - программа на языке B .

Классификация инвариантов

$$\varphi(i_k) \in \{L1, L2, L3, L4\}.$$

Модель защитного инварианта

Множество свойств безопасности (инвариантов безопасности) I_A задается следующим образом:

$$I_A = \{i_1, i_2, \dots, i_n\}, \quad (8)$$

где каждое свойство безопасности (инвариант безопасности) i_k — это функция над программой P_A (можно сказать, что это 3-значный предикат):

$$i_k: P_A \rightarrow \{0, 1/2, 1\}, \quad (9)$$

где:

- значение 0 означает, что соответствующее свойство безопасности имеет значение “не поддерживается”;
- значение 1/2 означает, что соответствующее свойство безопасности имеет значение “поддерживается частично”;
- значение 1 означает, что соответствующее свойство безопасности имеет значение “поддерживается полностью”.

Главное условие успешности по безопасности

$$|\{i_k \in I_A: \varphi(i_k) \in \{L1, L2\} \wedge i_k(P_A) \neq 0 \wedge i_k(P_B) = 0\}| = 0,$$

Метрика допустимого ослабления

$$\Delta(P_A, P_B) = |\{i_k \in I_A: \varphi(i_k) = L3 \wedge i_k(P_A) \neq 0 \wedge i_k(P_B) = 0\}|,$$

Условия сохранения семантики безопасности и функциональности

Семантика безопасности — это часть поведения и структуры программы, связанная с инвариантами безопасности, которые должны сохраняться, компенсироваться или контролируемо ослабляться при трансформации кода во время всего жизненного цикла программы.

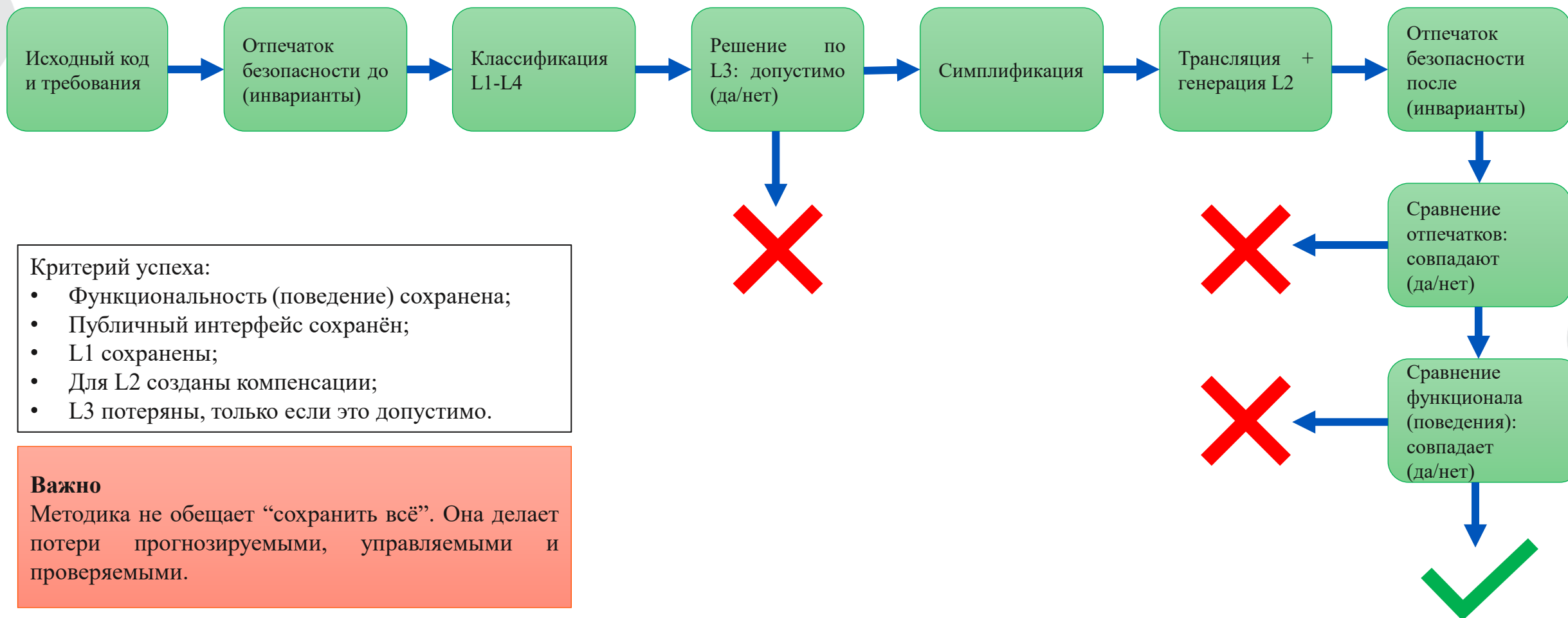
Проверяемый аспект	Условие успешности	Как проверяется
Функциональность	Результат работы не меняется	Одинаковые тестовые сценарии (для C# и Python)
Поведение	Одинаковые выходные значения и исключения	Сравнение выводов, исключений, последовательности вызовов
Публичный интерфейс	Публичный API сохраняется	Сопоставление классов, методов, сигнатур
Инварианты L1	Должны сохраниться	Сравнение отпечатка безопасности до и после
Инварианты L2	Должны быть созданы компенсации	Наличие дополнительного сгенерированного кода: проверки и т.д.
Инварианты L3	Допустимо теряются	Заранее фиксируются как допустимое ослабление
Инварианты L4	Могут появиться	Фиксируются как улучшение

Проверка функциональности через:

- ожидаемые сценарии использования;
- тестовые входные данные;
- сравнение результатов выполнения;
- сравнение исключений;

Публичные интерфейсы до и после трансляции сравниваются отдельно, т.к. могут быть определены разрешенные изменения

Методика контролируемой трансляции



Симплификация исходного кода

C#, характеристики Roslyn (The .NET Compiler Platform SDK):

- Количество разных типов узлов AST равно 241
- Количество разных значений перечисления SyntaxKind, определенных в библиотеке Roslyn, равно 561

Как итог – потенциально транслятору приходится учитывать до 561 вариантов SyntaxKind

Идея: уменьшить число разных случаев, которые должен корректно обработать транслятор.

Меньше вариантов AST → меньше риск дефекта транслятора → выше предсказуемость результата

Гипотеза - если симплификатор реализован корректно, то:

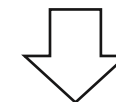
- правила симплификации могут быть спроектированы таким образом, чтобы сохранять семантику безопасности
- уменьшается вероятность ошибки/дефекта в трансляторе.

Важно

Симплификация не является "украшением" кода. Это подготовительный этап, уменьшающий сложность транслятора и риск искажения защитных инвариантов.

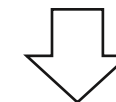
Цикл for на языке C#:

```
for (Int32 index = 0; index < 100; ++index)
{
    // loop body
}
```



Цикл while на языке C#:

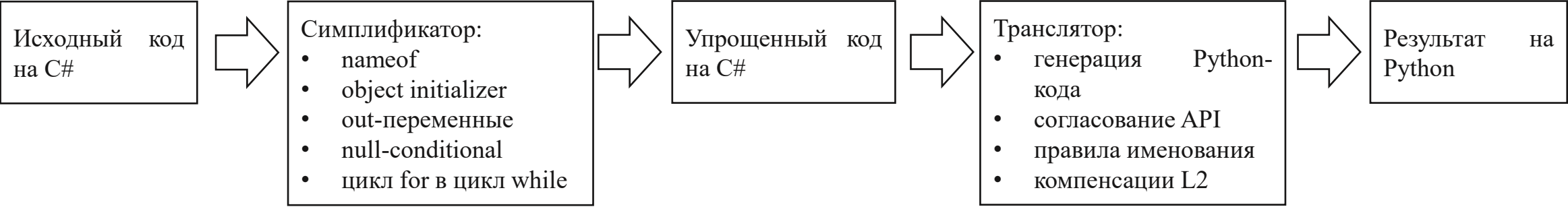
```
Int32 index = 0;
while (index < 100)
{
    // loop body
    ++index;
}
```



Цикл while на языке Python:

```
index = 0
while index < 100:
    # loop body
    index += 1
```

Реализация прототипа для трансляции с языка C# на язык Python



Компенсации для инвариантов безопасности L2

Инвариант безопасности	Компенсация
Потеря статической типизации	Проверки во время выполнения
Перегрузка методов	dispatch по числу и типам аргументов

Проблема

Создание библиотеки адаптера в языке Python имитирующей API .NET:

- в BCL (Base Class Library) содержится примерно 10000 - 15000 типов
- в ASP.NET Core содержится примерно 20000-30000 типов

Создание такой библиотеки - достаточно трудоемкий процесс

Ссылка на репозиторий симплификатора:



Ссылка на репозиторий транслятора:



Эксперимент: что проверялось

Объект эксперимента:

Модельный фрагмент C#-кода для графических примитивов:

- Объектная модель;
- Проверки корректности данных;
- Перегруженные конструкторы и методы добавления объектов;
- readonly-поля;
- Члены с модификаторами доступа protected/private;
- Получение строкового представления.

Контролируемые инварианты безопасности

Инвариант безопасности	Класс
проверки входных аргументов	L1
гарантия состояния при исключениях	L1
абстрактные типы и методы	L1
типобезопасность	L2
перегруженные методы	L2
инкапсуляция и модификаторы доступа	L3
readonly-поля	L3

Сравниваемые подходы

1. Трансляция по методике

2. Ручная трансляция

3. Трансляция с помощью LLM

Функциональные требования:

- Сохраняются создаваемые объекты;
- Сохраняется поведение методов GetRepresentation;
- Сохраняется порядок добавления шагов;
- Сохраняются исключения при некорректных данных;
- Сохраняется публичный интерфейс;
- Результаты программ на C# и Python совпадают на тестовых сценариях.

Ссылка на репозиторий:



Результаты сравнения: чем методика лучше

Критерий	Методика	Ручная трансляция	LLM трансляция	Вывод
Функциональное поведение	100%	100%	100%	Все подходы могут сохранить функциональное поведение
Публичный интерфейс	сохранен	изменен	изменен	Методика лучше контролирует API
Сохранено L1	3/3	3/3	2/3	LLM трансляция потеряла часть защитных свойств
Создано компенсаций L2	2/2	0/2	0/2	Только методика компенсирует потери языка
Потеряно L3	2/2, зафиксировано	2/2, неуправляемо	2/2, неуправляемо	Методика делает потери явными
Предсказуемость	высокая	зависит от разработчика	зависит от LLM и промпта	Методика более управляемая и явная

Эксперимент показал, что функциональная эквивалентность сама по себе не гарантирует сохранения семантики безопасности. Ручная и LLM-трансляция могут дать работающий код, но не обеспечивают контролируемое сохранение инвариантов L2 и публичного интерфейса. Предложенная методика обеспечивает проверяемое сохранение инвариантов L1, создание компенсаций для инвариантов L2 и фиксацию допустимых потерь инвариантов L3.

Итог, достижение цели, ограничения

Разработано:

- Модель отпечатка безопасности;
- Классификация инвариантов безопасности L1–L4;
- Условия сохранения семантики безопасности;
- Методика трансляции;
- Симплификатор исходного кода на C#;
- Транслятор с языка C# на язык Python;
- Экспериментальное сравнение с ручной трансляцией и трансляцией с помощью LLM.

Доказано экспериментально:

- Функциональность может сохраняться во всех подходах;
- Только методика сохраняет инварианты безопасности L1 и компенсирует инварианты безопасности L2;
- Потери инвариантов безопасности L3 являются не ошибкой, а ограничением целевого языка и должны фиксироваться заранее;
- Подход позволяет заранее прогнозировать изменение защитных свойств.

Цель работы достигнута: предложена и экспериментально проверена методика, позволяющая управляемо контролировать сохранение инвариантов защиты при трансляции программного кода между языками

Ограничения:

- Проверка была выполнена для трансляции с языка C# на язык Python;
- Отпечаток безопасности формировался вручную;
- Не введена весовая модель значимости инвариантов;
- Требуется расширение эксперимента на большее количество кодовых баз;
- Необходима автоматизация выделения инвариантов через AST/SAST/CWE/CERT.